TASK 3: WEB APPLICATION SECURITY TESTING

Practical Report with Screenshot Evidence | 11 October 2026

Prepared for the ApexPlanet Cybersecurity Internship — Task 3

Target	**DVWA on Kali Linux — http://127.0.0.1/DVWA/**
Tools	Kali Linux, DVWA, Apache, MariaDB, PHP, Burp Suite Community, curl
Scope	Local deliberately vulnerable training application only
Report status	Completed tests and limitations are stated separately

1. Objective

To practice identifying selected web application security weaknesses in a deliberately vulnerable application, test a basic SQL injection mitigation, and inspect HTTP traffic and response headers.

2. Lab Environment

Kali Linux virtual machine with Apache, MariaDB and PHP.

Target: Damn Vulnerable Web Application (DVWA), accessed locally at http://127.0.0.1/DVWA/.

Tools: Burp Suite Community Edition and curl.

DVWA security level was set to Low for demonstrations.

Scope: local training environment only; no external systems were targeted.

3. Methodology

Open the relevant DVWA module and observe its normal behaviour; submit controlled test input or inspect HTTP requests/responses; record the result; test a mitigation where implemented; and explicitly document unsuccessful or incomplete tests.

4. Experiments and Results

4.1 SQL Injection

Normal input 1 returned the admin record. The input ' OR '1'='1' # returned multiple user records in DVWA at Low security.

Result: SQL injection was demonstrated in the intentionally vulnerable module.

Risk: unsafe query construction can allow user input to change query logic or retrieve unintended records.

4.2 SQL Injection Mitigation

A separate local file, /var/www/html/DVWA/secure-sqli.php, was created using a MySQLi prepared statement with a placeholder and bind_param().

Input 1 returned the expected admin record. The injection string ' OR '1'='1' # did not return all user records.

Result: the prepared-statement example resisted the tested input. Use parameterized queries rather than concatenating untrusted input into SQL.

4.3 Cross-Site Scripting (XSS)

Stored XSS: submitted <script>alert('Stored XSS')</script>; the browser displayed the alert.

Reflected XSS: submitted <script>alert('Reflected XSS')</script>; the browser displayed the alert.

DOM XSS: a controlled payload on the DVWA DOM-XSS page displayed the “DOM XSS” alert.

Risk: injected script may execute in a visitor’s browser in the vulnerable site’s context.

Mitigation: context-aware output encoding, safe DOM APIs, and a restrictive Content Security Policy as an additional layer.

4.4 File Inclusion

The normal File 1 content loaded. A path-traversal attempt targeting /etc/hostname did not display the file; Apache/PHP logs reported that the path could not be opened.

PHP showed allow_url_include = Off, and DVWA reported remote URL inclusion was not enabled.

Result: the attempted file disclosure was unsuccessful. This report does not claim successful file disclosure.

Mitigation: allowlist page identifiers, map them to fixed files, constrain paths, and never pass raw request parameters directly to include().

4.5 Burp Suite HTTP Traffic Inspection

Burp Proxy HTTP history displayed local DVWA requests, including GET requests to /DVWA/ and /DVWA/vulnerabilities/sqli/, as well as login-related requests.

Result: Burp captured local application traffic. HTTP history alone does not prove that an attack succeeded.

4.6 Cryptography Module

The DVWA Cryptography module encoded “hello” as “HwQPBbs=” and decoded “HwQPBbs=” back to “hello”.

Result: the encode/decode round trip worked. Encoding is not encryption and does not provide confidentiality by itself.

4.7 API Module

The DVWA API page was opened and verified to load. The page displayed warnings about mod_rewrite and Composer-related setup.

Result: only page availability was checked; a full API vulnerability assessment was not completed.

4.8 HTTP Security Headers

curl inspection of the local DVWA URL showed HTTP/1.1 302 Found, a Server header identifying Apache/2.4.68 (Debian), and Set-Cookie headers.

The filtered check returned no matching lines for X-Frame-Options, X-Content-Type-Options, Content-Security-Policy, Strict-Transport-Security or Referrer-Policy in the inspected response. These headers were not observed in that response; this is not a complete review of all endpoints or redirect responses.

Recommendation: configure suitable headers and retest the final authenticated response over HTTPS where applicable.

5. Incomplete Tests and Limitations

CSRF: successfully demonstrated in the local DVWA lab at Low security. While authenticated, a crafted link triggered a password change using a state-changing GET request. This illustrates the risk of missing or ineffective CSRF protections. The test was limited to the local lab.

JavaScript Attacks: the module was tested, but the attempted token was rejected as invalid; incomplete.

File Inclusion: traversal did not disclose /etc/hostname; remote inclusion was disabled.

API Security: only page loading was verified.

Security headers: inspection was limited to the local response and selected header names.

6. Key Findings

SQL injection was reproduced in DVWA at Low security; the separate prepared-statement example prevented the tested injection from returning all records.

Stored, reflected and DOM XSS test payloads triggered browser alerts in the local lab.

Burp Suite captured local DVWA HTTP requests.

The File Inclusion traversal attempt did not succeed.

The inspected response exposed an Apache server banner, and the selected common security headers were not observed in the filtered output.

7. Recommendations

Use parameterized SQL queries and least-privilege database accounts.

Encode untrusted output for its HTML/JavaScript/URL context and avoid unsafe DOM sinks.

Use fixed allowlisted page identifiers for file inclusion.

Protect state-changing actions with validated anti-CSRF tokens and appropriate cookie settings.

Configure and verify Content-Security-Policy, X-Content-Type-Options, frame protections, Referrer-Policy and HSTS when using HTTPS.

Reduce unnecessary server-version disclosure and keep the web stack patched.

Repeat testing across security levels and inspect authenticated responses as well as redirects.

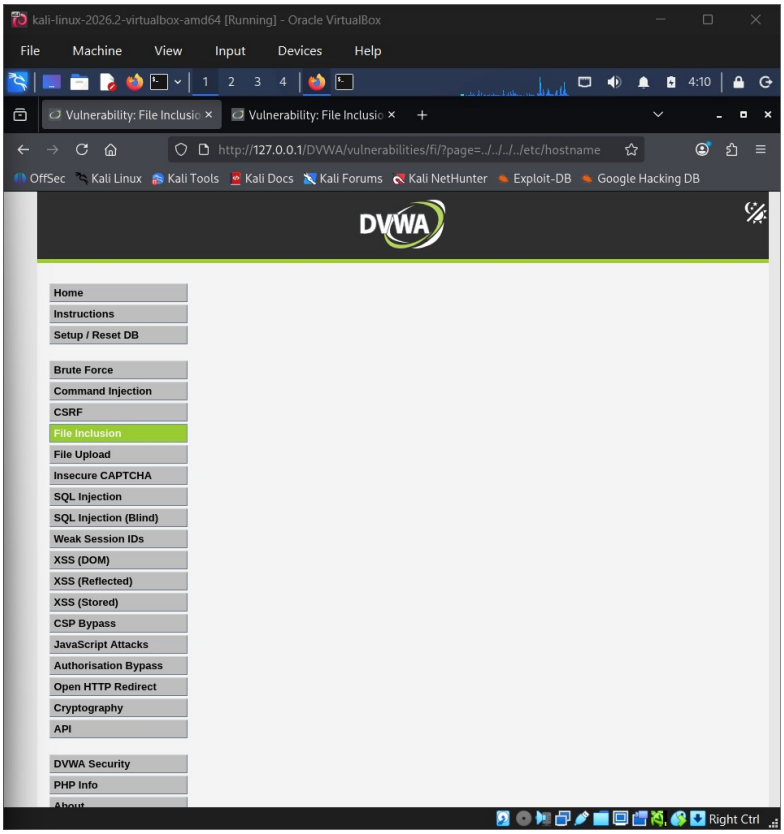
8. Conclusion

This practical demonstrated selected web security concepts using a deliberately vulnerable local application. SQL injection and three XSS variants were reproduced, a prepared-statement mitigation was tested, and HTTP traffic and selected response headers were inspected. Incomplete activities and unsuccessful tests are identified explicitly. All work was restricted to the local training lab.

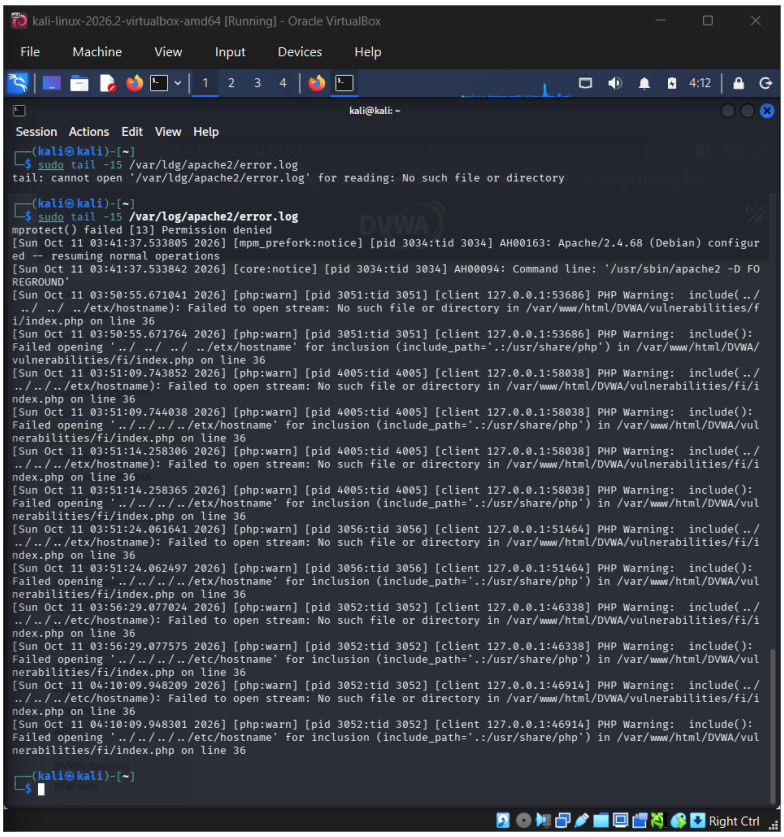
Appendix A — Screenshot Evidence

The following screenshots were attached during the Task 3 session. They are included as submitted and have not been altered. Use the experiment descriptions above to interpret each item; a screenshot is evidence of the visible state, not necessarily proof that a vulnerability was successfully exploited.

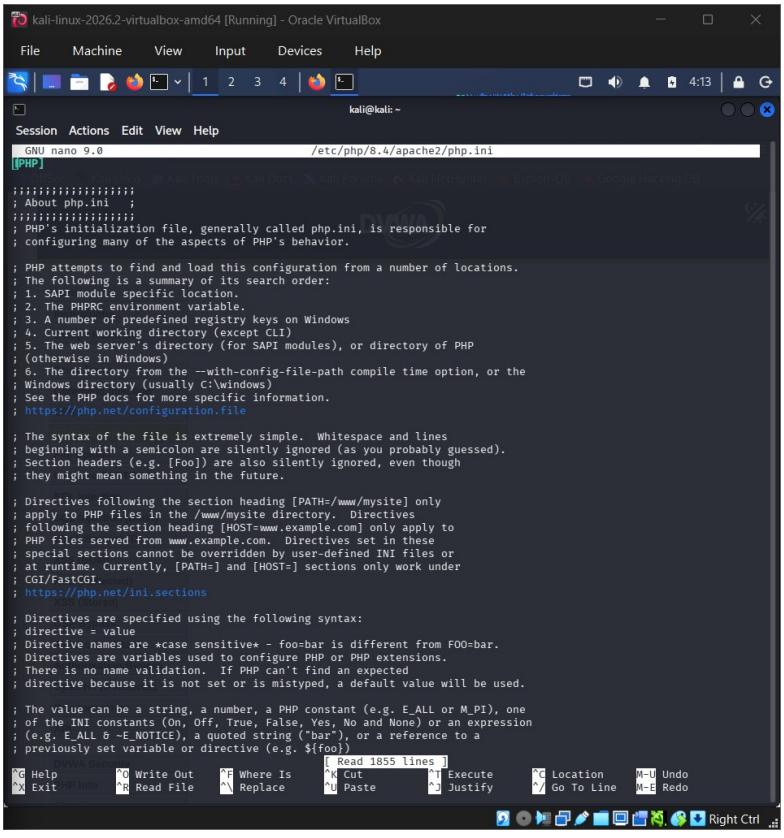
Evidence 01: Screenshot 2026-10-11 134021.png



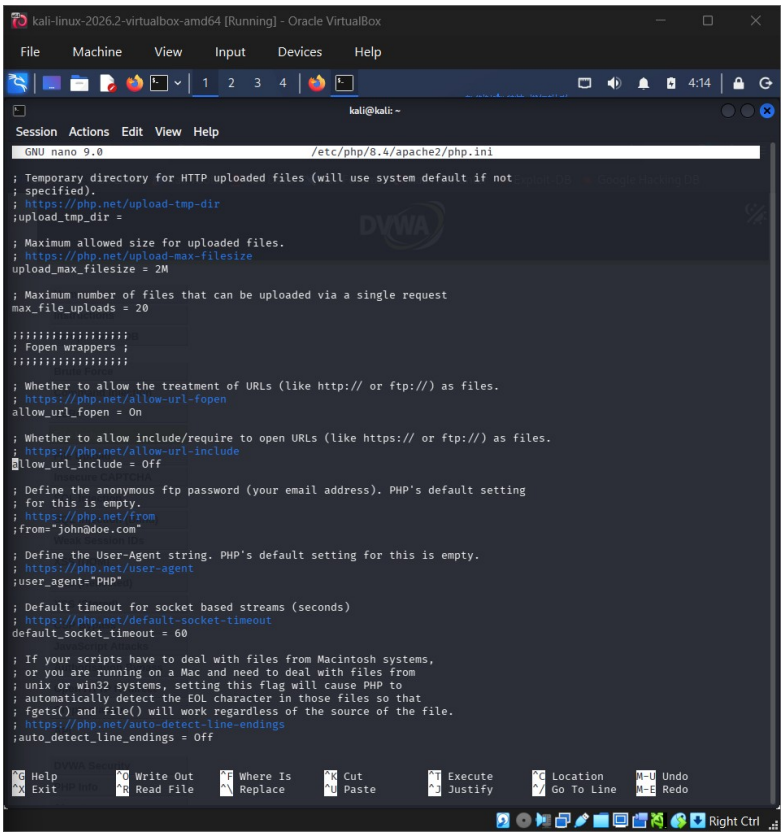
Evidence 02: Screenshot 2026-10-11 134207.png



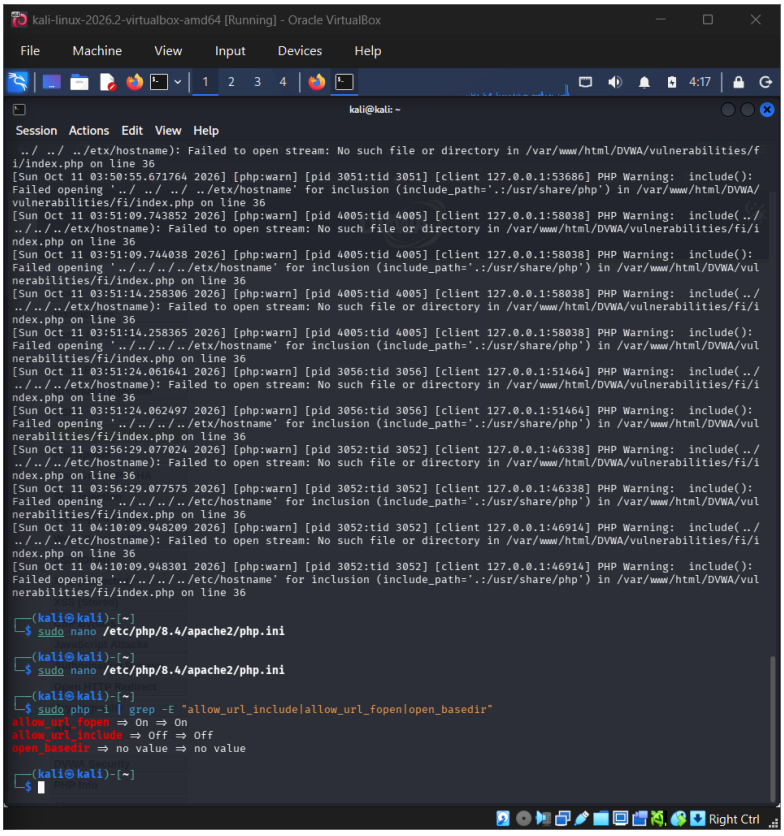
Evidence 03: Screenshot 2026-10-11 134315.png



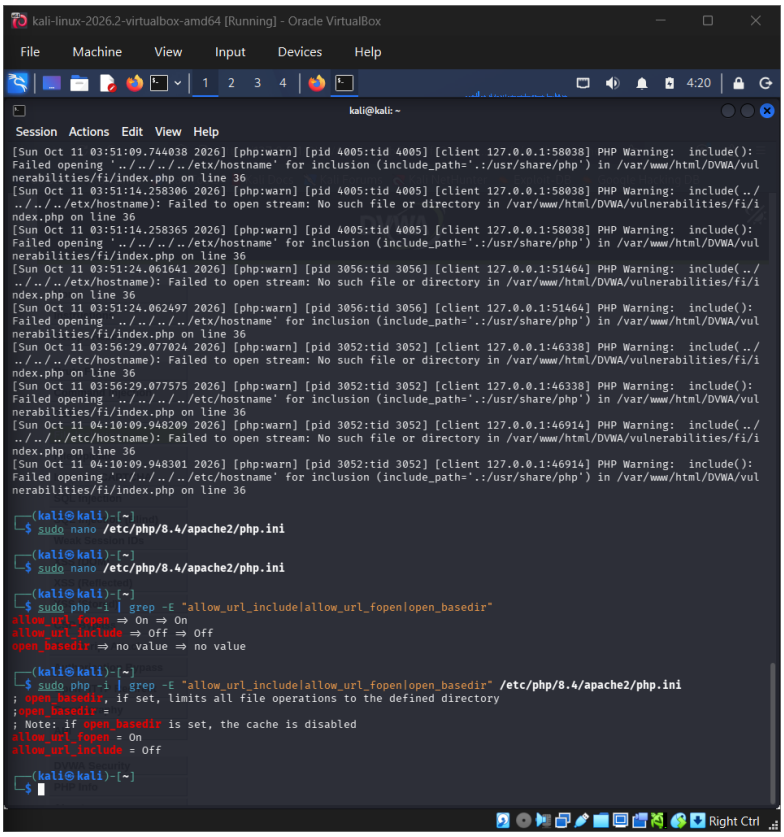
Evidence 04: Screenshot 2026-10-11 134421.png



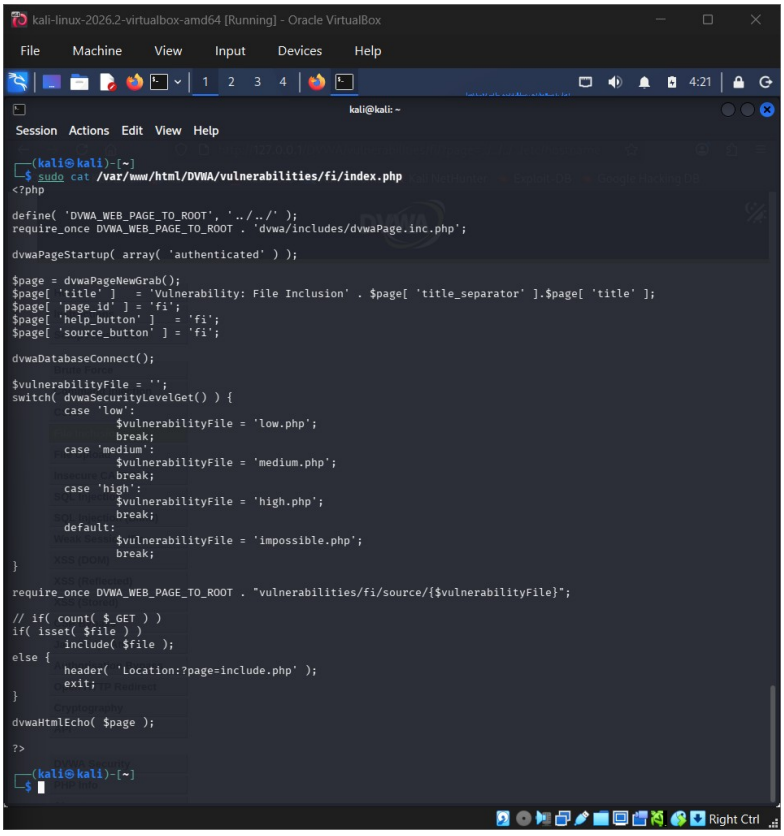
Evidence 05: Screenshot 2026-10-11 134712.png



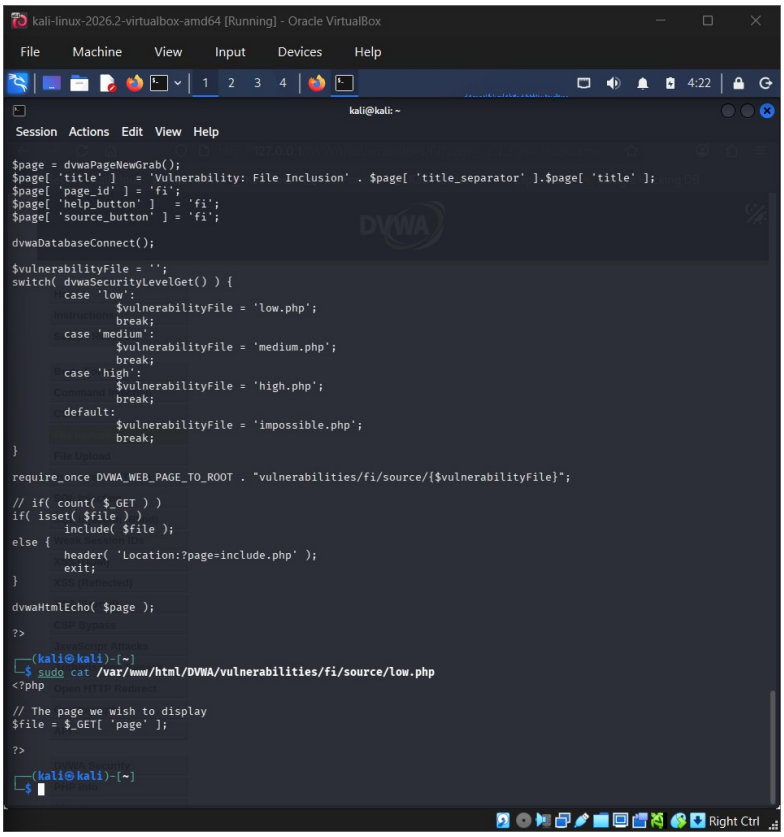
Evidence 06: Screenshot 2026-10-11 135013.png



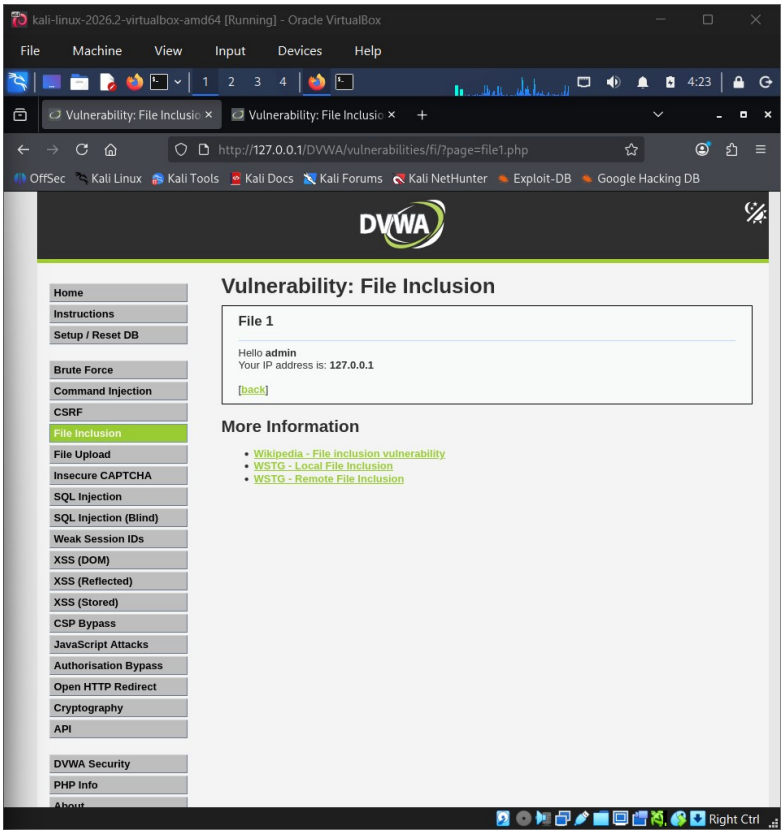
Evidence 07: Screenshot 2026-10-11 135109.png



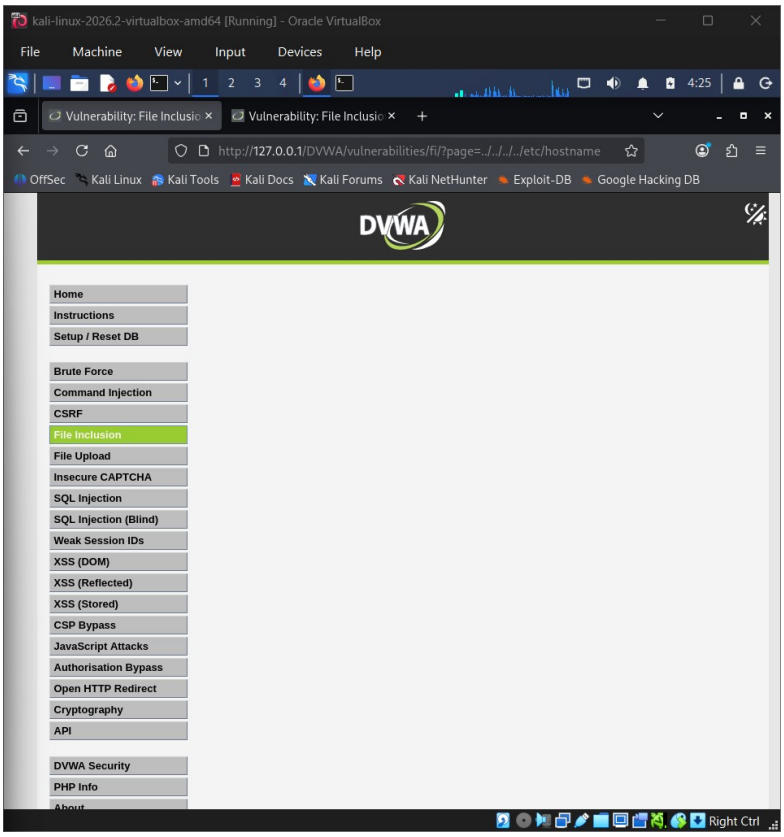
Evidence 08: Screenshot 2026-10-11 135205.png



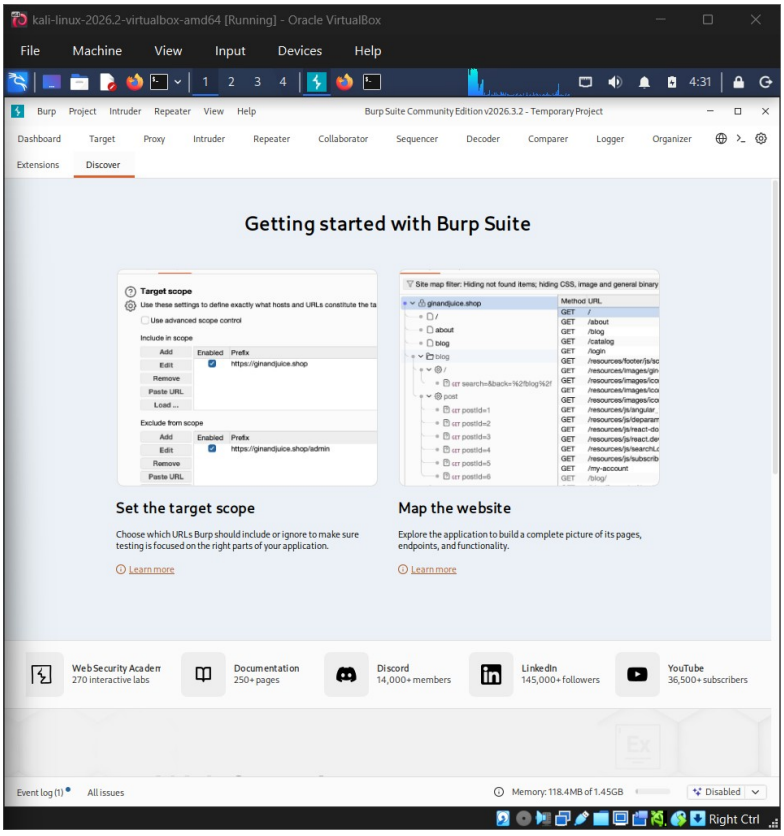
Evidence 09: Screenshot 2026-10-11 135351.png



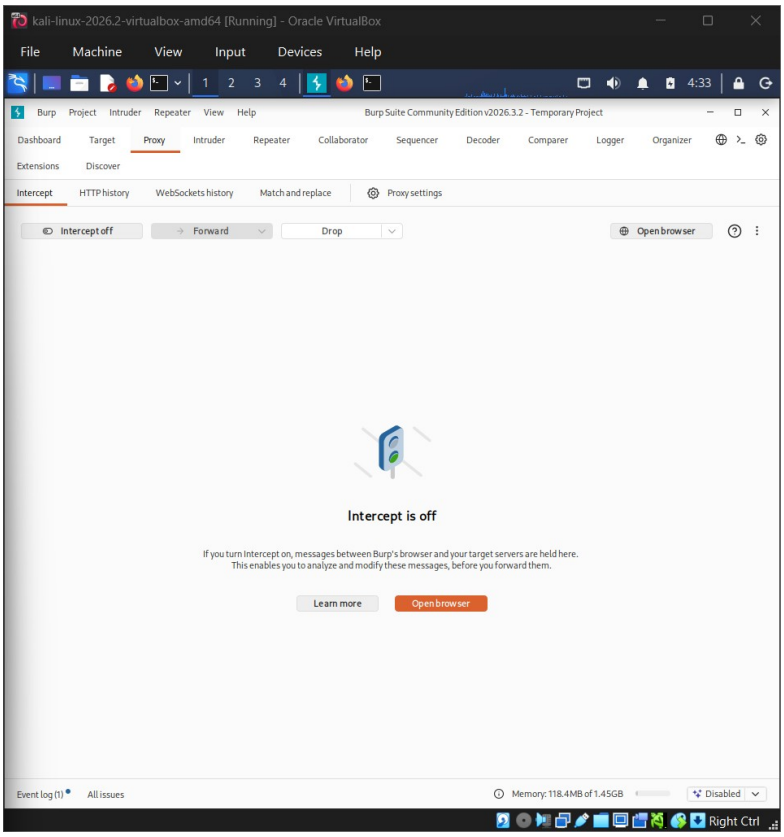
Evidence 10: Screenshot 2026-10-11 135549.png



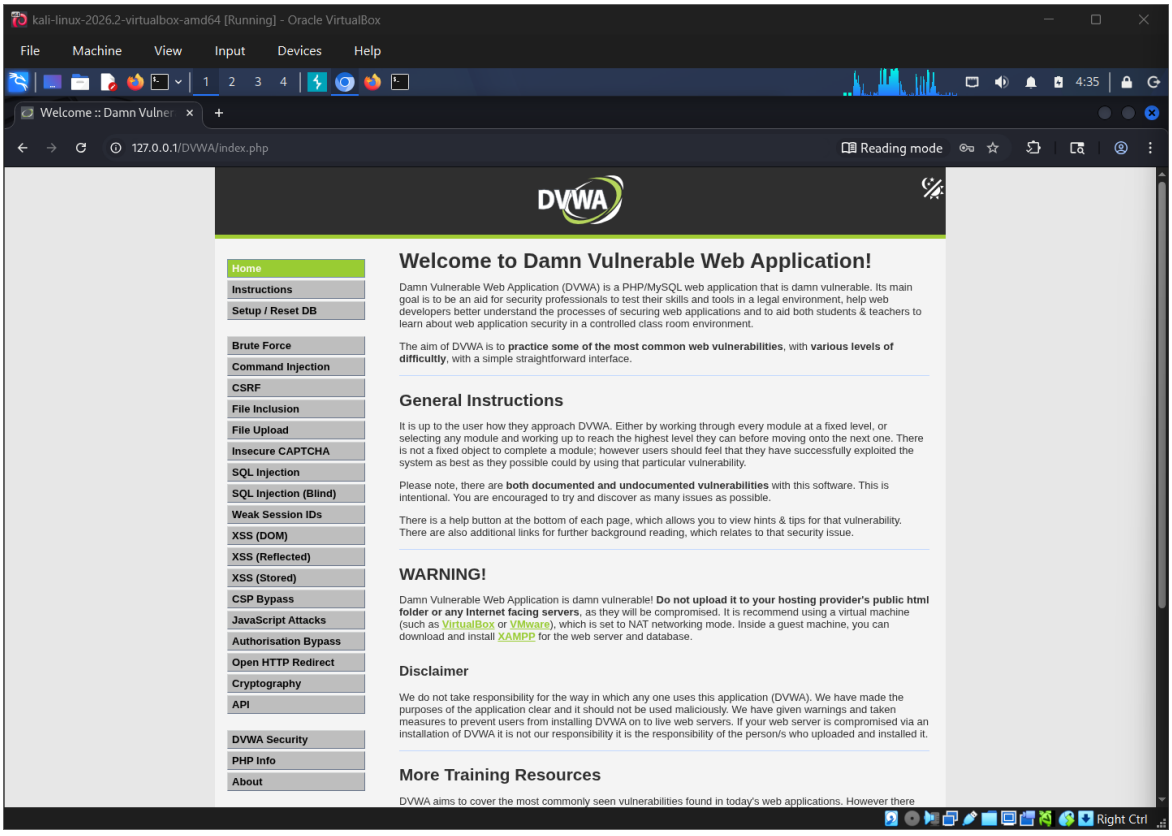
Evidence 11: Screenshot 2026-10-11 140123.png



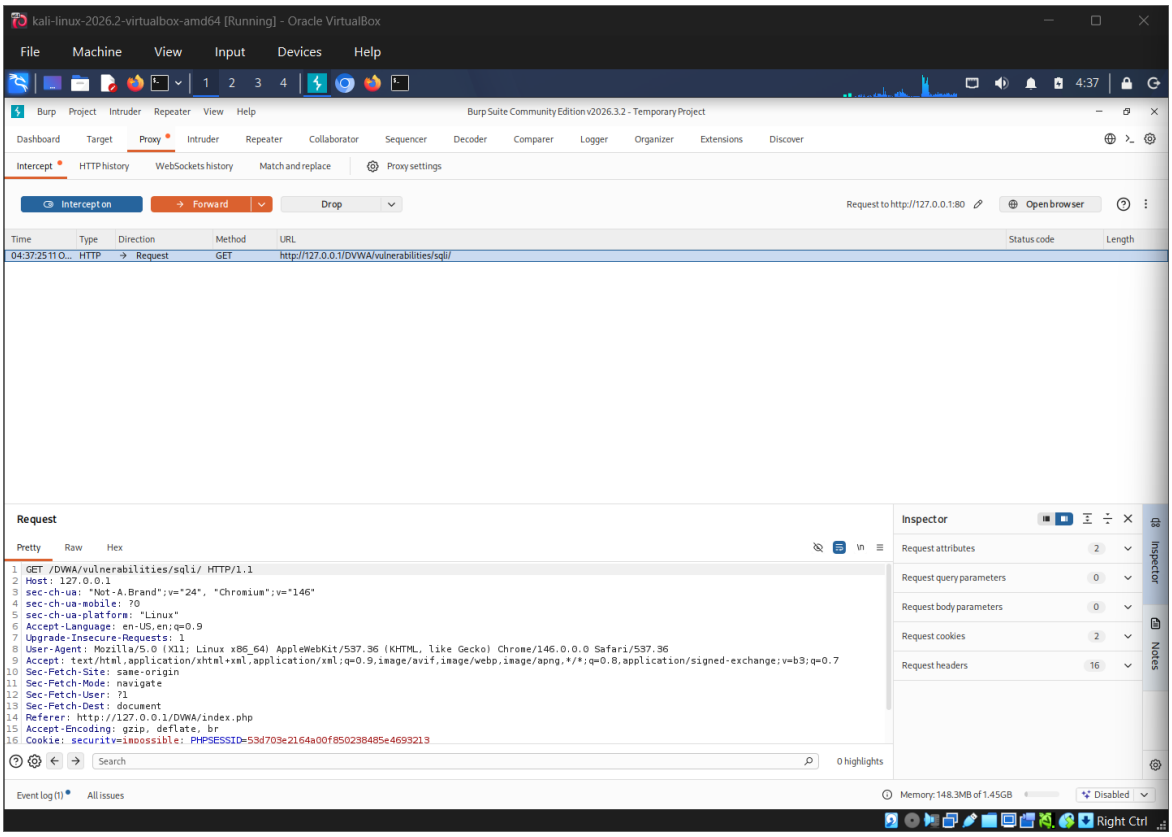
Evidence 12: Screenshot 2026-10-11 140305.png



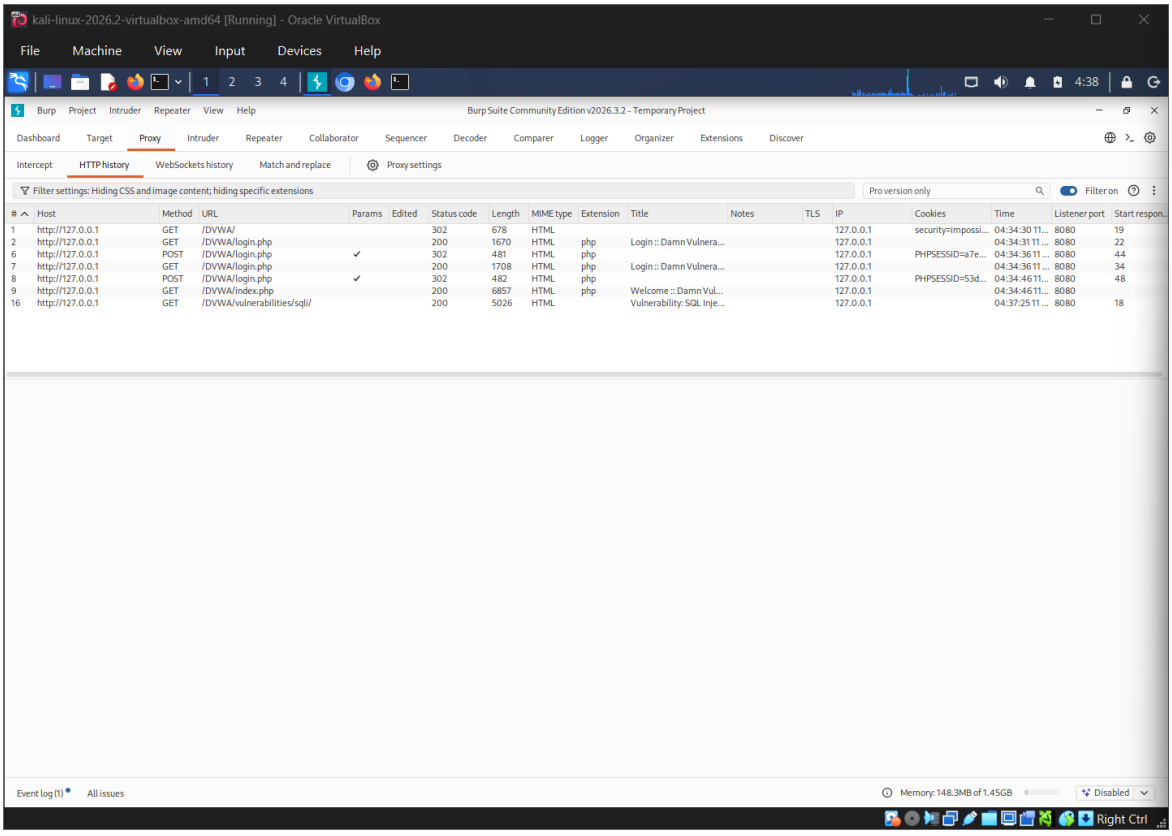
Evidence 13: Screenshot 2026-10-11 140602.png



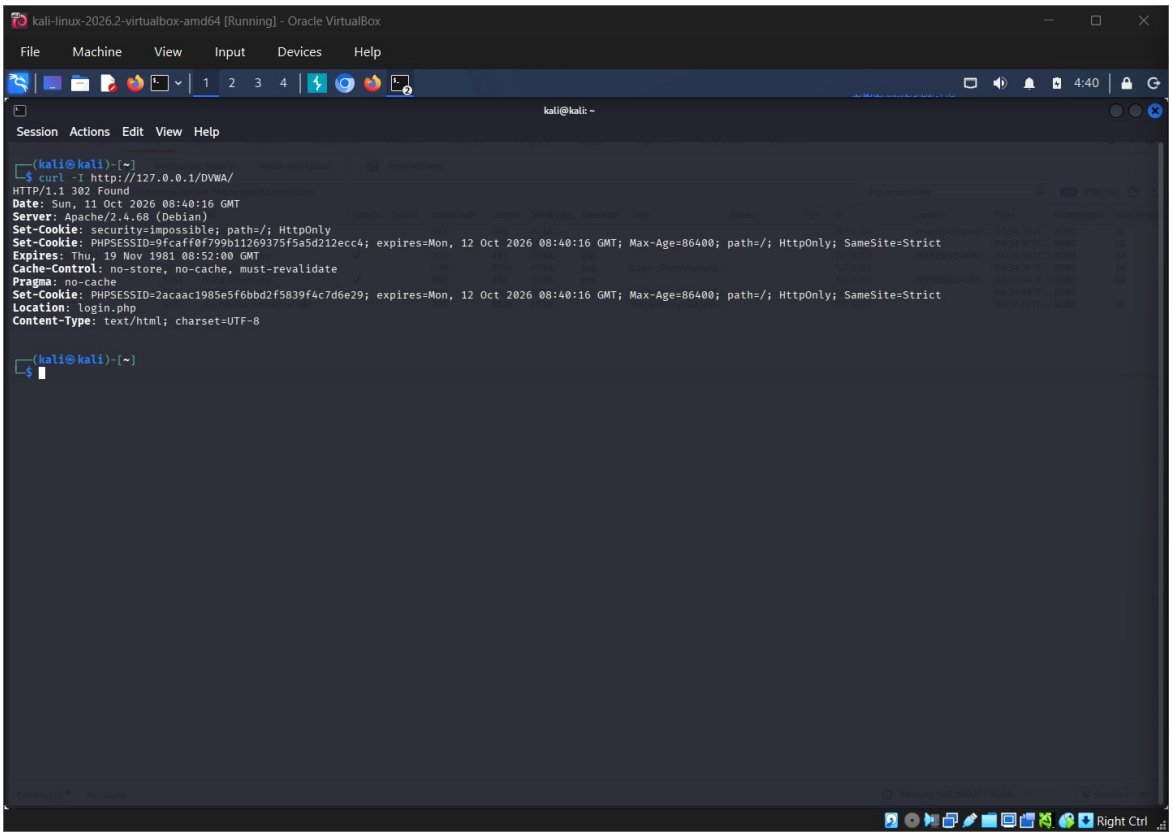
Evidence 14: Screenshot 2026-10-11 140738.png



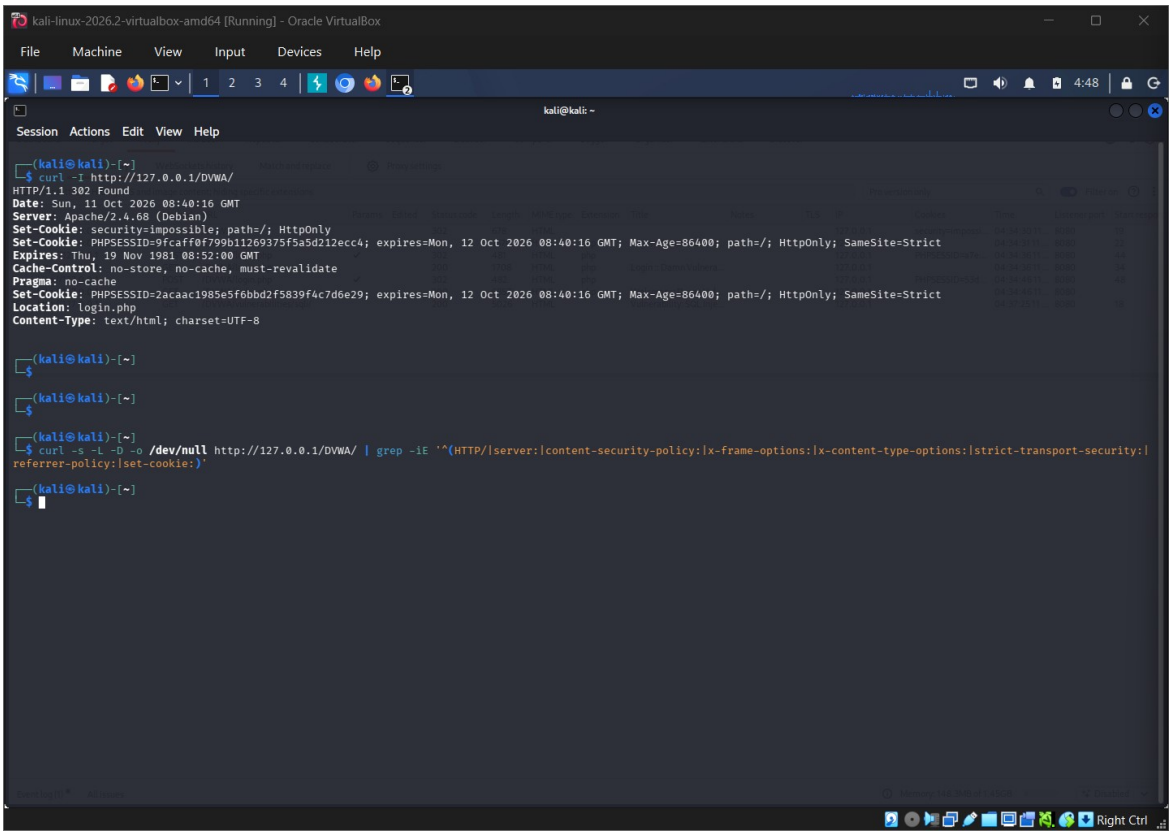
Evidence 15: Screenshot 2026-10-11 140852.png



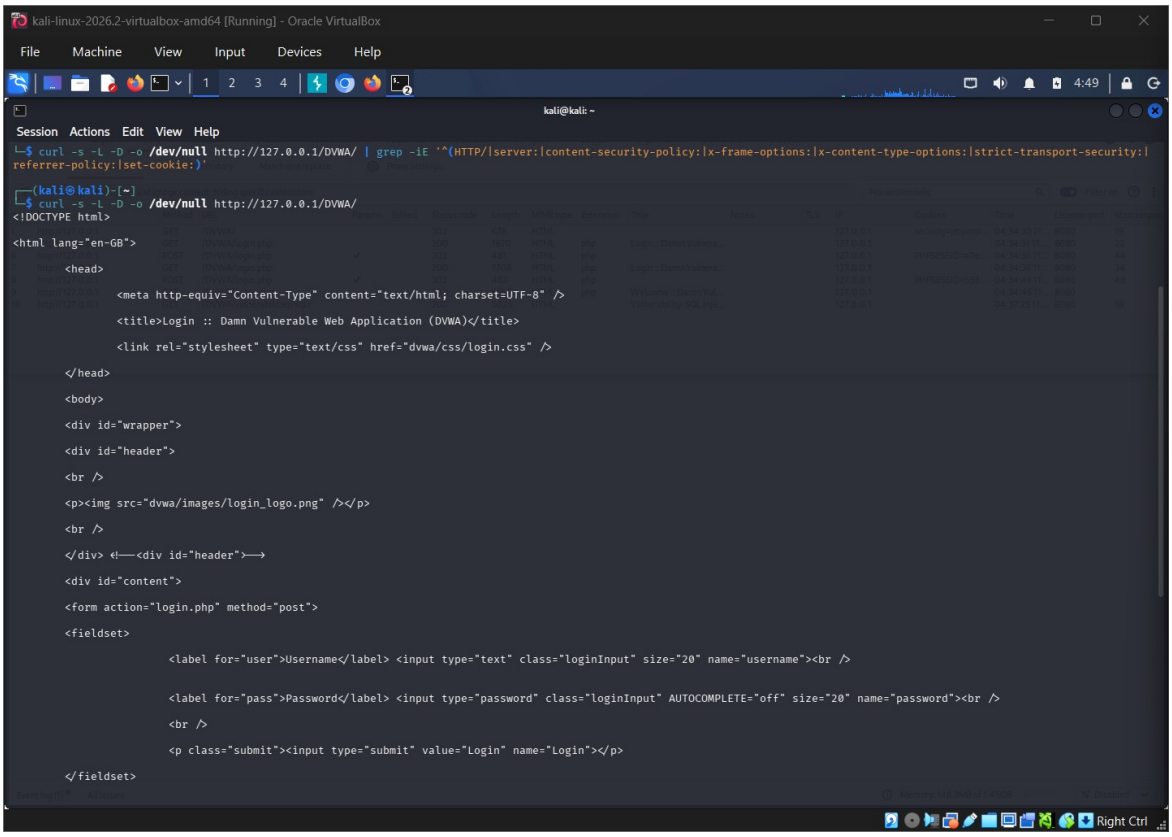
Evidence 16: Screenshot 2026-10-11 141022.png



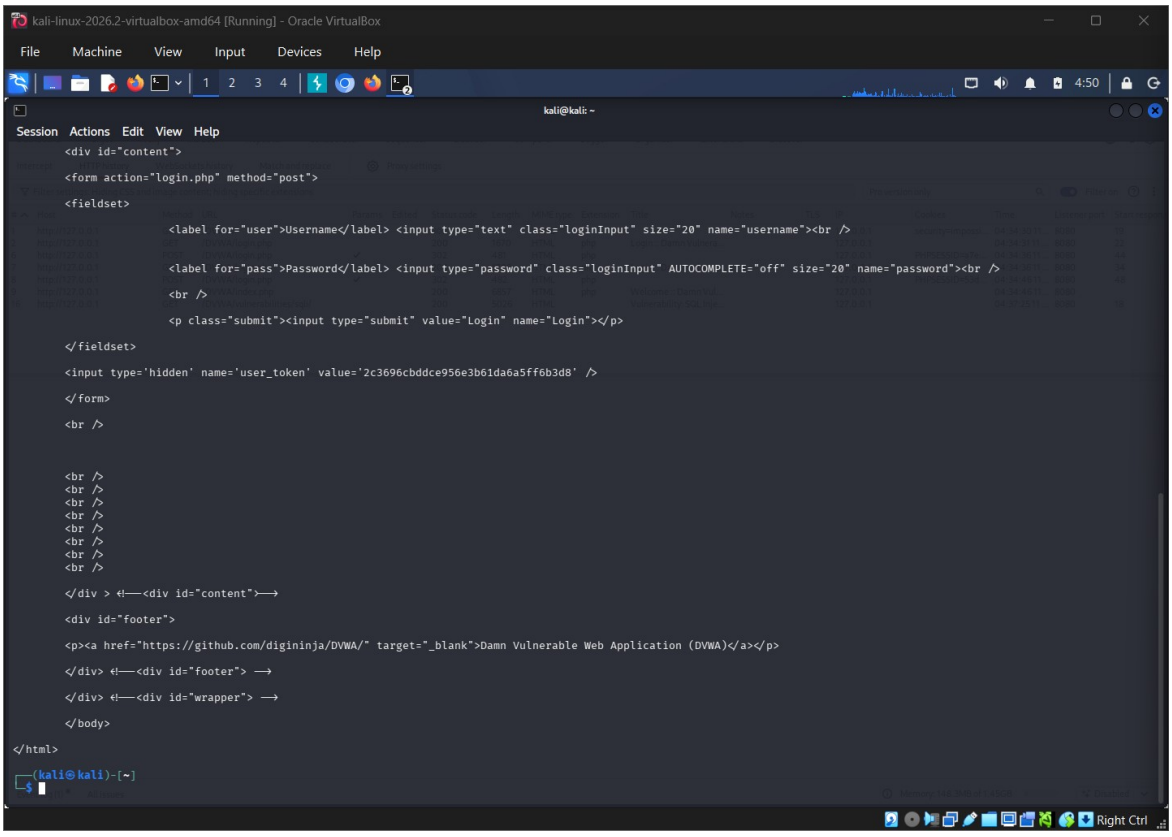
Evidence 17: Screenshot 2026-10-11 141804.png



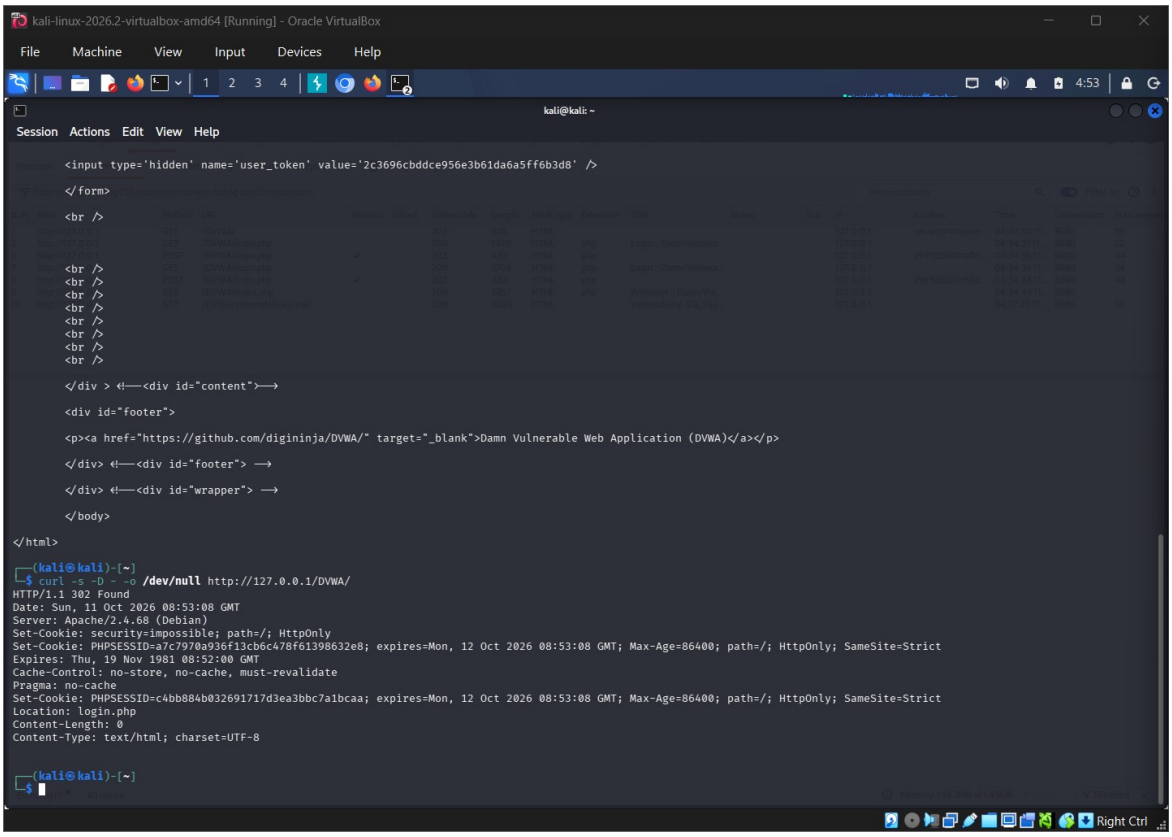
Evidence 18: Screenshot 2026-10-11 141956.png



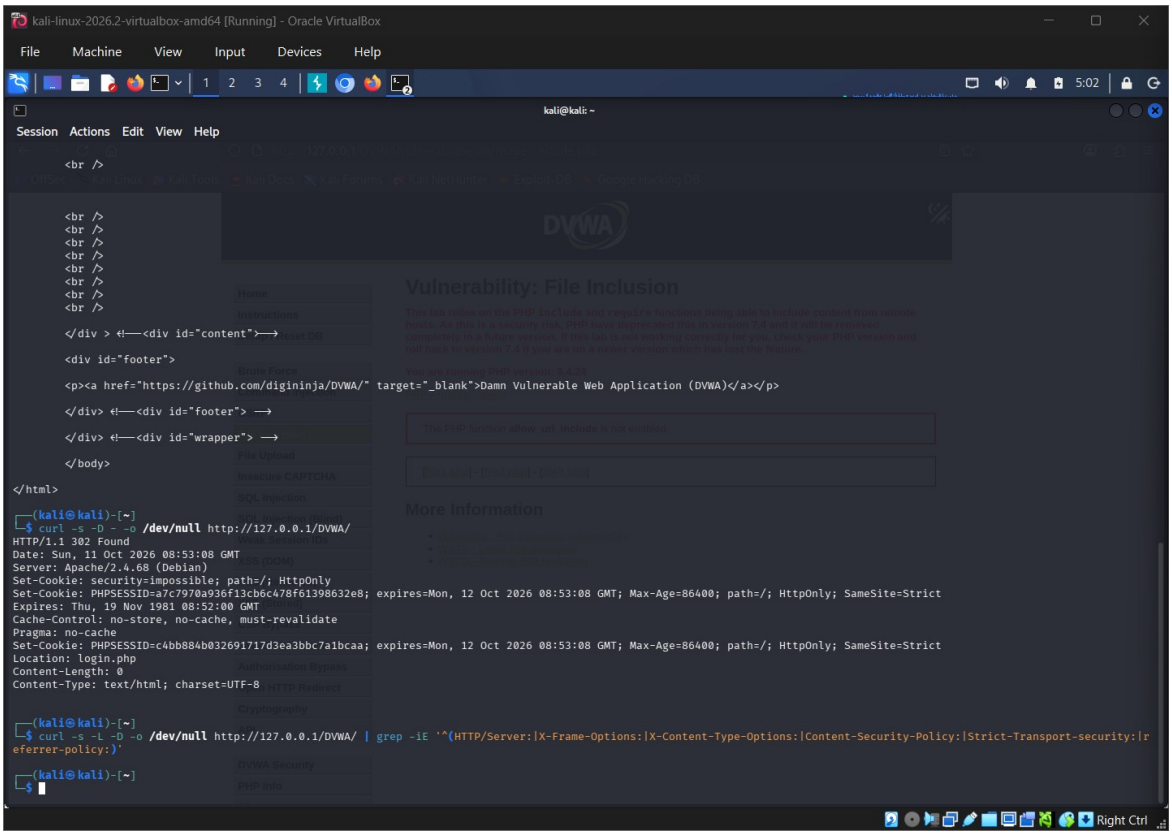
Evidence 19: Screenshot 2026-10-11 142008.png



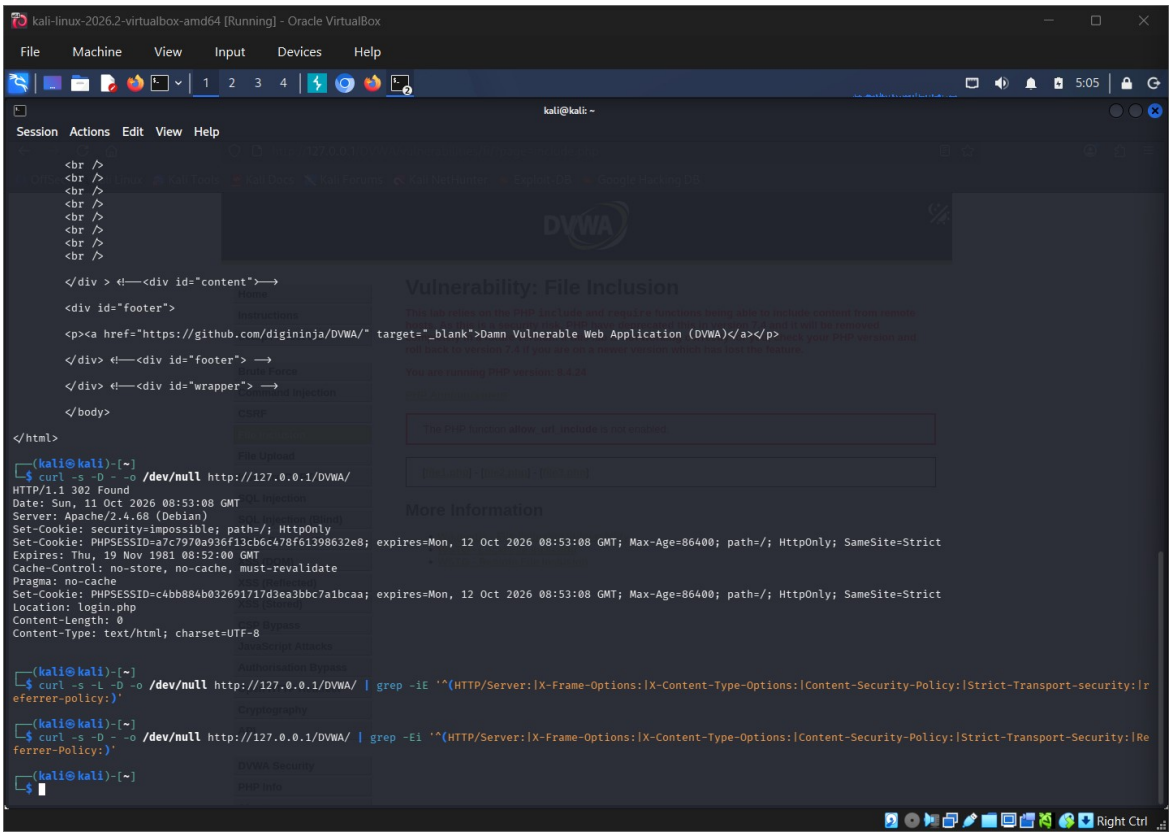
Evidence 20: Screenshot 2026-10-11 142313.png



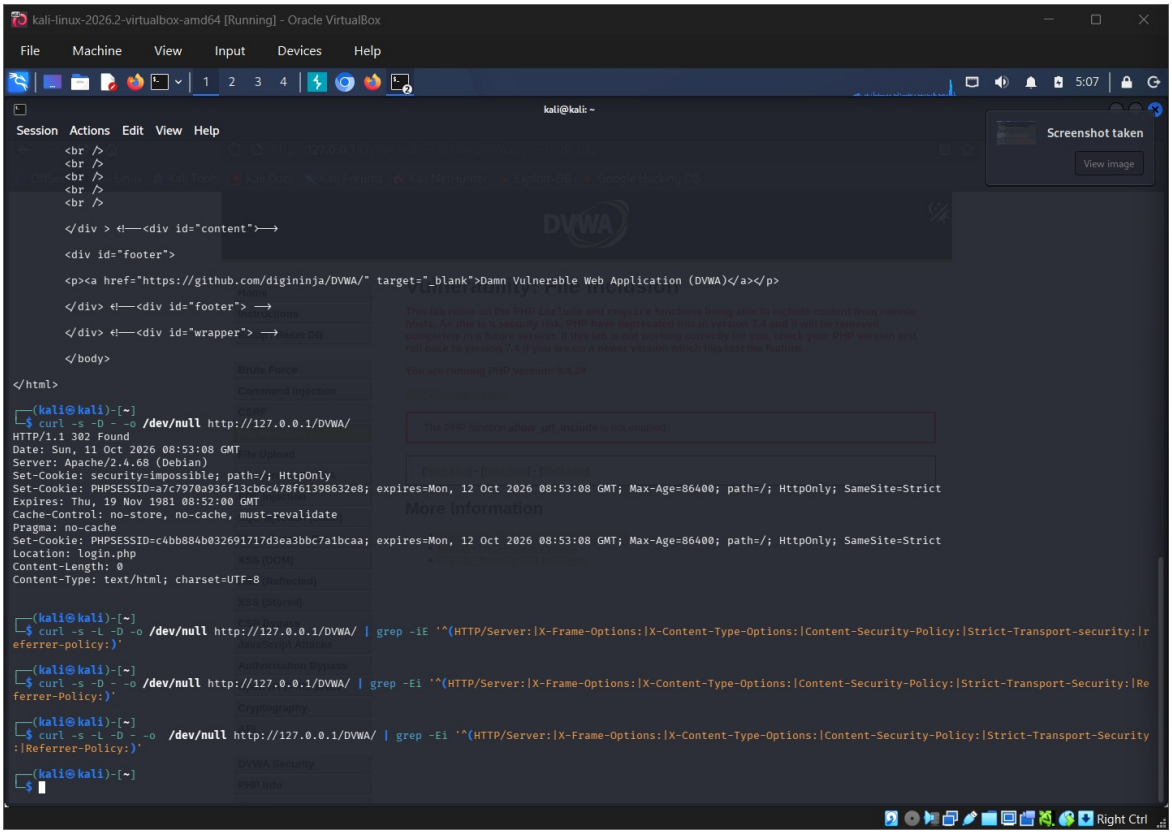
Evidence 21: Screenshot 2026-10-11 143234.png



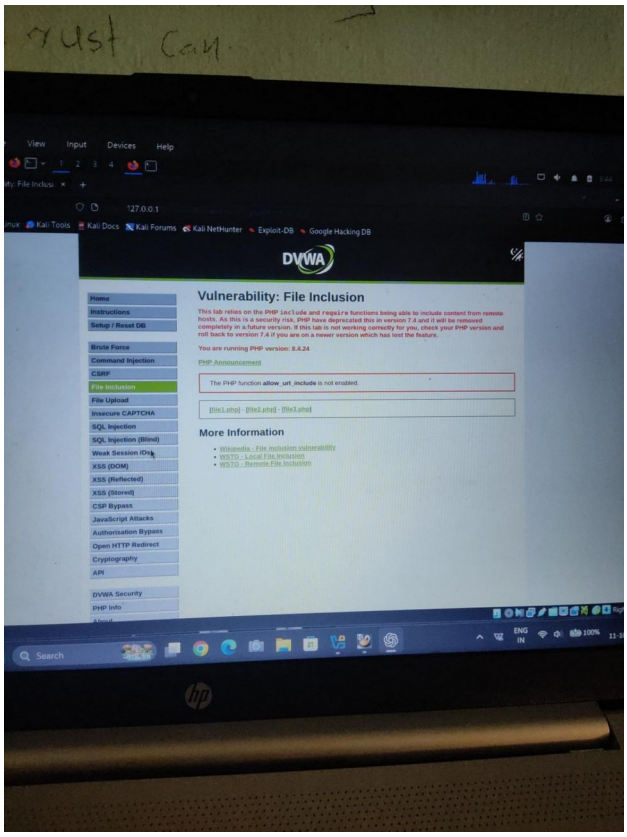
Evidence 22: Screenshot 2026-10-11 143600.png



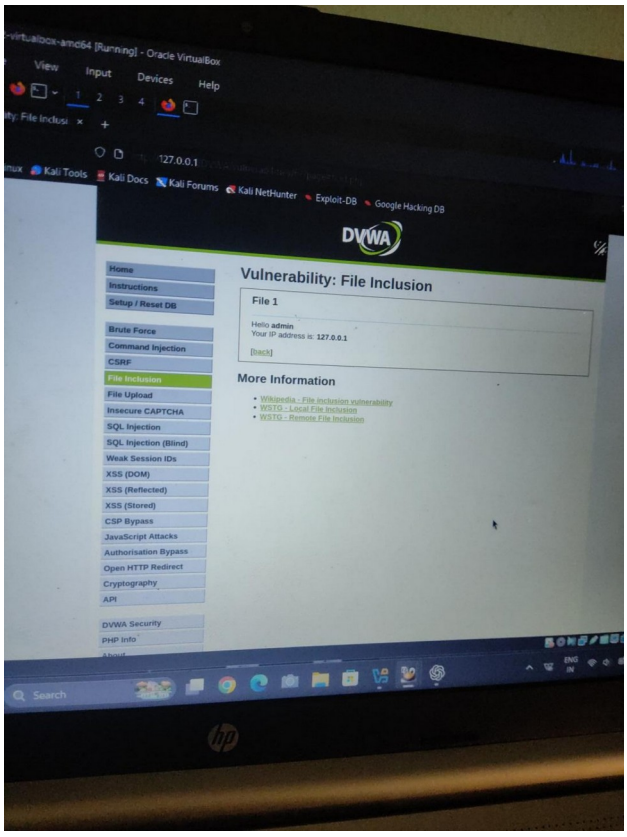
Evidence 23: Screenshot 2026-10-11 143742.png



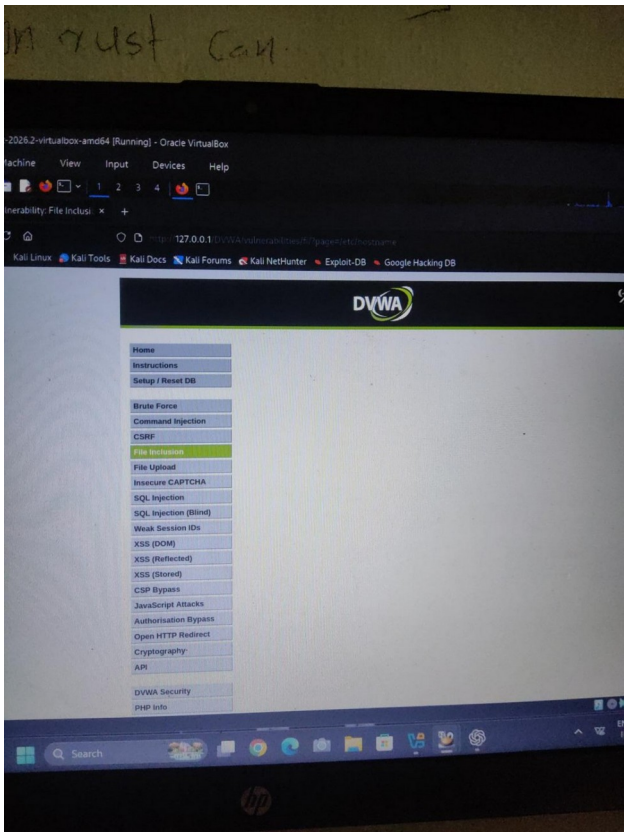
Evidence 24: image-1791704645964.jpg



Evidence 25: image-1791704708436.jpg



Evidence 26: image-1791704819159.jpg



Evidence 27: image-1791706033566.jpg

